

LiteRAG: Cost-Efficient Graph-Based Retrieval-Augmented Generation

Daniel Alejandro Coll Tejeda Pedro García López Daniel Barcelona-Pons

Universitat Rovira i Virgili

{danielalejandro.coll, pedro.garcia, daniel.barcelona}@urv.cat

Abstract

Graph-based retrieval can improve multi-hop question answering, but existing approaches often incur high query-time costs and produce diffuse, oversized contexts that reduce generation efficiency. We present LiteRAG, a graph-based retrieval method that replaces expensive retrieval-time LLM control with query-conditioned algorithmic exploration and reasoning-chain context construction. On DistComp, a benchmark for multi-hop retrieval over distributed-systems papers, LiteRAG attains the highest overall quality among the evaluated methods (0.798) while reducing per-query latency by over $100\times$ and cost by over 99% relative to GraphRAG Global and DRIFT. On UltraDomain, it matches LinearRAG on overall quality while using about $14\times$ fewer tokens. An ablation study indicates that LiteRAG’s query-adaptive thresholding and community-aware hub penalization are the main drivers of its token-efficiency gains.

1 Introduction

Retrieval-Augmented Generation (RAG) has become a standard approach for grounding Large Language Models (LLMs) in external knowledge (Lewis et al., 2020; Fan et al., 2024). In its most common form, RAG retrieves text chunks by dense similarity and passes them to the generator as supporting context. This design is effective for many lookup-style queries, but it remains weak on questions that require combining evidence across documents, tracing relational dependencies, or synthesizing information over multiple hops (Gao et al., 2023; Jiang et al., 2023b).

Graph-based RAG methods address this limitation by representing corpora as structured graphs in which entities, relations, and communities are explicitly modeled (Edge et al., 2024; Guo et al., 2025; Huang et al., 2025a; Zhuang et al., 2026). However, many existing systems still rely heavily on the LLM at query time to traverse the

graph, summarize communities, or aggregate evidence across retrieved regions. This dependence increases latency and cost, scales poorly with corpus size, and can produce broad contexts that force the generator to sift through substantial irrelevant or weakly relevant information (Liu et al., 2024).

For multi-hop question answering, the challenge is therefore not only to retrieve enough evidence, but to present it in a form that preserves a high density of query-relevant information. When retrieval returns loosely filtered text or broad summaries, the generator must still deduce how entities connect. Under realistic token budgets, this can dilute useful evidence and reduce the value of structured retrieval.

We therefore argue that graph retrieval for RAG should optimize both evidence selection and how retrieved evidence is prepared for the generator. We introduce LiteRAG, a method that removes the generation model from the retrieval loop. Rather than relying on sequential LLM calls for navigation, LiteRAG performs query-conditioned algorithmic graph exploration: it first selects semantic and lexical anchor nodes, then expands a bounded subgraph using dynamic thresholding and community-aware hub penalization.

The final stage addresses the same objective at the context level. Rather than passing large text collections or broad summaries to the generator, LiteRAG converts the retrieved subgraph into compact reasoning chains that express the retained relations directly in the prompt. This yields a more compact, query-focused context in which relevant connections are already explicit, preserving the structure needed for generation under restricted token budgets.

Our contributions are summarized as follows:

- Query-conditioned algorithmic retrieval: LiteRAG replaces fixed-hop or LLM-mediated traversal with algorithmic exploration based

on dynamic thresholding and community-aware hub penalization.

- Reasoning-chain context construction: LiteRAG converts the retrieved subgraph into compact reasoning chains, producing a more information-dense context that remains effective within a budget of about 2,000 tokens.
- Quality-efficiency gains: On DistComp, LiteRAG attains the highest overall quality and lowest latency and cost among the evaluated methods; relative to the most expensive LLM-intensive GraphRAG configurations, it reduces per-query cost by over 99% and latency by over 100 $\times$.

To support reproducibility, the source code for LiteRAG, benchmark queries, configurations, and evaluation scripts will be released on GitHub.

2 Related Work

Recent work on Retrieval-Augmented Generation (RAG) (Fan et al., 2024) has shifted from flat passage retrieval toward structured augmentation to better support compositional multi-hop reasoning. Within this trend, graph-based methods explicitly encode entities and relations, enabling retrieval over relational structure rather than isolated text chunks.

Graph-based RAG methods can be grouped into two broad lines. A first line uses the graph to organize evidence but still relies heavily on the LLM at retrieval time. Microsoft GraphRAG (Edge et al., 2024) is the clearest example: LLMs are used both upstream to help construct graph representations and, more importantly, at query time to orchestrate traversal, summarize communities, and retrieve evidence from the graph. Related graph-aware reasoning systems such as G-Retriever (He et al., 2024) and Think-on-Graph (Sun et al., 2024) likewise show the value of explicit relational structure. Across this line of work, the graph improves retrieval quality, but retrieval-time control remains closely coupled to the LLM.

A second line aims to make graph-based RAG more efficient. Among the most relevant systems for our setting, LightRAG (Guo et al., 2025) and HiRAG (Huang et al., 2025a) improve retrieval through dual-level or hierarchical organization, while related structured retrieval approaches such as RAPTOR (Sarathi et al., 2024) reorganize evidence before generation. More algorithmic meth-

ods such as HippoRAG (Gutiérrez et al., 2024), LinearRAG (Zhuang et al., 2026), KET-RAG (Huang et al., 2025b), and ROGRAG (Wang et al., 2025) reduce LLM dependence through graph scoring, indexing, and filtering, but they often rely on static graph statistics or fixed heuristics and give less direct attention to constructing a dense final context for the generator.

LiteRAG belongs to this second line, but differs from prior systems in both evidence collection and final context construction. Instead of relying on fixed neighborhoods, static graph statistics, or retrieval-time LLM decisions, it performs query-conditioned subgraph construction and then converts the retained evidence into compact reasoning chains. Whereas methods such as LightRAG, HiRAG, and LinearRAG primarily improve retrieval organization or graph scoring, LiteRAG couples adaptive evidence collection with explicit construction of a more information-dense final context. Relative to Microsoft GraphRAG and related LLM-mediated systems, retrieval-time control is shifted away from the LLM.

3 LiteRAG Method

LiteRAG is a graph-based RAG method for cost-efficient retrieval over knowledge graphs. It replaces retrieval-time LLM control with query-conditioned algorithmic exploration and explicit context construction before final answer generation. As shown in Figure 1, the method has three stages: (1) *query-conditioned anchor selection*, which identifies promising graph entry points; (2) *query-conditioned subgraph expansion*, which builds a relevant subgraph from those anchors using semantic and structural signals; and (3) *reasoning-chain context construction*, which converts the retrieved subgraph into a compact representation for the generator.

3.1 Phase 1: Query-Conditioned Anchor Selection

Given a query q and a knowledge graph $G = (V, E)$, Phase 1 selects an anchor set $A \subset V$ of semantically and lexically salient entities. Rather than scoring every node in V , LiteRAG retrieves a candidate pool $V_c \subset V$ via top- K vector search, assigning each $v \in V_c$ a composite score $S(v)$:

$$S(v) = \alpha S_{\text{sem}}(q, v) + \beta S_{\text{lex}}(q, v) + \gamma S_{\text{com}}(q, C(v)) \quad (1)$$

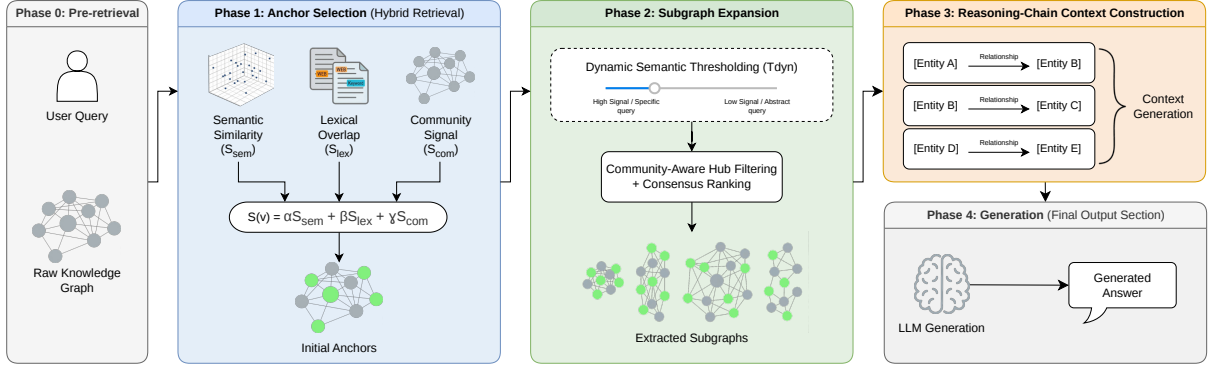


Figure 1: Overview of the LiteRAG method.

where $C(v)$ denotes the community containing v , and α , β , and γ are non-negative weights that balance the three signals. The components of the score are: $S_{\text{sem}}(q, v)$ is the semantic similarity between the query representation and node v , $S_{\text{lex}}(q, v)$ is a lexical matching score designed to preserve entities that may be crucial by name, such as proper nouns or domain-specific terms, even when semantic similarity alone is insufficient, and $S_{\text{com}}(q, C(v))$ is a community-level relevance signal that favors nodes in graph regions whose aggregate content is aligned with the query.

This formulation reflects the retrieval objective of the phase: maximize the chance that exploration starts from informative graph regions without over-committing to a single notion of relevance. The semantic term captures conceptual alignment, the lexical term protects exact terminology, and the community term biases the search toward coherent topical neighborhoods.

After computing $S(v)$ for all candidate nodes in V_c , LiteRAG constructs the anchor set through thresholding:

$$A = \{v \in V_c \mid S(v) \geq \tau_{\text{anchor}}\} \quad (2)$$

where τ_{anchor} is the anchor threshold. The resulting set A serves as the initial frontier for Phase 2, where LiteRAG expands a query-conditioned subgraph from these entry points.

3.2 Phase 2: Query-Conditioned Subgraph Expansion

Phase 2 takes the anchor set A from Phase 1 and the knowledge graph $G = (V, E)$, and returns a query-conditioned subgraph $G_q = (V_q, E_q)$ expanded from those anchors. Expansion proceeds from all anchors in parallel. Rather than delegating retrieval-time navigation to an LLM, LiteRAG

treats graph traversal as an algorithmic search problem in which candidate expansions are accepted or pruned according to a query-conditioned relevance score.

Concretely, LiteRAG performs a parallel subgraph expansion initialized at the nodes in A . At each step, the current frontier contains nodes already admitted into the explored region. For a frontier node u and one of its neighbors v , LiteRAG computes a traversal relevance score that determines whether v should be incorporated into the explored subgraph and added to the next frontier. Expansion therefore grows adaptively around the anchors, and the final output is the subgraph induced by the retained nodes and traversed edges. Exploration stops when no frontier nodes satisfy the relevance criterion below, when the maximum hop depth k_{max} is reached, or when the per-anchor expansion cap N_{max} is exhausted.

3.2.1 Query-Adaptive Thresholding

The first component of the exploration rule is a query-conditioned threshold that controls how selective traversal should be for a given query. Instead of using a global fixed threshold, LiteRAG derives a dynamic threshold from the quality of the initial anchors:

$$\tau_{\text{dyn}}(q, A) = \min \left(1.0, \tau_{\text{base}} + \lambda \max_{a \in A} S(a) \right) \quad (3)$$

where τ_{base} is a minimum relevance floor, λ controls how strongly anchor evidence influences the threshold, and $S(a)$ is the anchor score from Phase 1; the outer minimum caps τ_{dyn} at 1.0 because $S(a)$ is an uncalibrated composite score rather than a strict probability. This formulation links exploration directly to the quality of the retrieval starting points: strong anchors induce a stricter traversal

criterion, while weaker anchors allow broader exploration of potentially relevant graph regions.

3.2.2 Community-Aware Hub Penalization

The second component of the exploration rule addresses high-degree hubs, which can connect otherwise unrelated graph regions and cause the search to drift. To reduce this effect, LiteRAG penalizes candidate expansions through nodes whose structural centrality is more likely to reflect generic connectivity than query relevance.

For a frontier node u , a neighboring candidate node v , and hop depth k , LiteRAG defines the traversal relevance as:

$$R(q, u, v, k) = S_{\text{sem}}(q, v) \cdot d^k \cdot P_{\text{hub}}(u, v) \quad (4)$$

where $S_{\text{sem}}(q, v)$ is the semantic similarity between the query and node v , d is a depth-decay factor, and $P_{\text{hub}}(u, v)$ is a hub penalty. A candidate expansion is retained when:

$$R(q, u, v, k) \geq \tau_{\text{dyn}}(q, A) \quad (5)$$

The hub penalty is defined as:

$$P_{\text{hub}}(u, v) = \frac{1}{1 + \delta \log(1 + \deg(v))\omega(u, v)} \quad (6)$$

where $\deg(v)$ is the degree of node v , δ controls the strength of the hub penalty, and $\omega(u, v)$ modulates this penalty according to community alignment:

$$\omega(u, v) = 1 - \mathbb{I}(C(u) = C(v))\kappa \quad (7)$$

Here, $\mathbb{I}(\cdot)$ is the indicator function, $C(u)$ and $C(v)$ denote the communities of u and v , and $\kappa \in [0, 1]$ controls how strongly within-community transitions are protected from hub penalization. As a result, high-degree nodes that remain within the same topical region are penalized less aggressively than hubs that bridge unrelated communities.

Together, the dynamic threshold and hub-aware penalty define the expansion rule used to grow the frontier from the anchors. The output of this phase is the retained query-conditioned subgraph G_q , which is then passed to Phase 3 for ranking and context construction.

3.3 Phase 3: Reasoning-Chain Context Construction

Phase 3 converts the query-conditioned subgraph G_q into the final generation context. It aims to preserve relational evidence while expressing it in a compact form under strict token budgets.

To do so, LiteRAG ranks entities in G_q and retains relations incident to the top-ranked ones for inclusion in the final reasoning-chain context. For each explored entity $v \in V_q$, LiteRAG computes a consensus score $S_{\text{cons}}(v)$ that combines four normalized signals:

$$S_{\text{cons}}(v) = \rho_1 I_{\text{rate}}(v) + \rho_2 S_{\text{sem}}(q, v) + \rho_3 P_{\text{prox}}(v) + \rho_4 S_{\text{str}}(v) \quad (8)$$

where $I_{\text{rate}}(v)$ is the fraction of anchors whose expansions reach v , $S_{\text{sem}}(q, v)$ is the semantic alignment between v and the query, $P_{\text{prox}}(v)$ is a topological proximity term that decays with the average hop distance from the anchors, and $S_{\text{str}}(v)$ is a structural centrality score derived from graph measures such as PageRank and betweenness. To reward evidence supported by consistently relevant traversal paths, LiteRAG further rescales this score by the average traversal relevance accumulated when reaching v and by a small bonus when v is itself an initial anchor. Entities with the highest resulting scores are retained, and the relations connecting them are converted into a compact context representation that exposes relational structure directly to the generator.

The main representation used in this phase is a *reasoning chain*, which linearizes a graph relation into a short relational statement rather than a larger unstructured text block:

$$\text{Entity A} \xrightarrow{[\text{Relationship}]} \text{Entity B: "Description of the specific interaction..."}$$

The final generation context combines these reasoning chains with minimal supporting information needed for interpretation, such as definitions of top-ranked entities or brief source-grounded evidence when available. In this way, LiteRAG converts the explored subgraph into a structurally explicit context that is substantially more compact than raw-text or community summaries.

This stage is related to *context compression* approaches (Jiang et al., 2023a), but its role here is more specific: rather than shortening arbitrary text, LiteRAG constructs a curated, information-dense relational context in which the relevant connections are already explicit.

Section 5.4 revisits this design choice and shows that the context-construction step reduces token consumption while preserving the explicit relational evidence needed for multi-hop reasoning.

4 Experimental Methodology

We evaluate LiteRAG on DistComp, a custom benchmark designed to test the query types and corpus-scaling behavior central to our setting, and on the *Mix* split of UltraDomain, a public benchmark evaluated under the same model setup across representative graph-based and dense-retrieval baselines.

4.1 Datasets

DistComp. We construct *DistComp* from Distributed Computing papers published between 2020 and 2022 in nine major IEEE and ACM venues. The benchmark is designed to cover query types that are less well represented in standard public datasets but central to our setting, especially multi-hop synthesis and drift-style comparison, and to test how retrieval behavior changes as corpus size grows. We create five corpus sizes, $D = \{40, 80, 160, 640, 1280\}$, with the largest split containing approximately 500,000 tokens, and annotate 160 expert-written queries in four categories: literal citation, local reasoning, global thematic synthesis, and drift-style multi-hop comparison across distant graph regions.

UltraDomain. We also evaluate on the *Mix* split of UltraDomain, a public multi-domain benchmark spanning agriculture, law, and healthcare.

4.2 Baselines

We compare LiteRAG against five baseline families: **GraphRAG Basic** as a dense-retrieval baseline over text chunks; **Microsoft GraphRAG** (Edge et al., 2024) in its *Local*, *Global*, and *DRIFT* modes; **LightRAG** (Guo et al., 2025) in *Local*, *Global*, *Mix*, and *Hybrid* modes; **HiRAG** (Huang et al., 2025a) in *Hi*, *Local*, *Global*, *Bridge*, and *No-Bridge* modes; and **LinearRAG** (Zhuang et al., 2026) in its default configuration.

4.3 Implementation Details

All methods use gemini-2.5-flash-lite for generation and gemini-embedding-001 for embeddings. LiteRAG hyperparameters were selected by grid search on a validation split and are reported in Appendix D. For graph construction, LiteRAG reuses Microsoft GraphRAG indexing (Edge et al., 2024) and differs from prior methods only in retrieval and context construction.

4.4 Evaluation Metrics

We report one composite quality metric and three efficiency measures. Overall quality is defined as

$$Q_{\text{total}} = 0.6 \cdot Q_{\text{judge}} + 0.25 \cdot Q_{\text{sem}} + 0.15 \cdot Q_{\text{lex}} \quad (9)$$

where Q_{judge} is an LLM-as-a-judge score, Q_{sem} is embedding-based semantic similarity, and Q_{lex} is ROUGE-L. The weights were fixed on a held-out validation split to emphasize reasoning quality while retaining semantic and lexical grounding; disaggregated values and the final weighting are reported in Appendix B. For efficiency, we report end-to-end latency in seconds per query, total token consumption, and cost per query under a shared API pricing model (\$0.10 per 1M input tokens and \$0.40 per 1M output tokens).

5 Results and Empirical Analysis

This section reports LiteRAG’s results in terms of quality, latency, token usage, and cost.

5.1 Comparative Performance on DistComp and UltraDomain

Table 1 reports the main comparison on the largest DistComp setting, D_{1280} , and the *Mix* split of UltraDomain. We report overall quality, Q_{total} , together with the three efficiency metrics from Section 4.4.

On D_{1280} , LiteRAG attains the highest overall quality ($Q_{\text{total}} = 0.798$), followed by LinearRAG (0.784) and HiRAG Local (0.777). It also records the lowest latency (1.42s), token usage (2,291), and cost per query (\$0.0003). The efficiency difference is largest against LLM-intensive methods: GraphRAG Global requires 162.66s and 643k tokens per query, while GraphRAG DRIFT exceeds 142s and 6.4M tokens.

The comparison with LinearRAG is especially informative because both methods reduce retrieval-time LLM reliance, but they do so through different retrieval pipelines. LinearRAG relies on transformer-based query-time entity extraction and iterative graph-ranking stages, whereas LiteRAG uses bounded query-conditioned expansion and reasoning-chain context construction. On D_{1280} , LiteRAG improves overall quality (0.798 vs. 0.784) while reducing latency from 8.01s to 1.42s, token usage from 3,077 to 2,291, and cost from \$0.0006 to \$0.0003 per query. This comparison suggests that LiteRAG can match or improve

Method	Configuration	DistComp D_{1280}				UltraDomain Mix			
		$Q_{\text{total}} \uparrow$	Lat. (s) $\downarrow$	Tokens $\downarrow$	Cost (\$) $\downarrow$	$Q_{\text{total}} \uparrow$	Lat. (s) $\downarrow$	Tokens $\downarrow$	Cost (\$) $\downarrow$
LiteRAG	—	0.798	1.42	2,291	0.0003	0.801	1.24	4,862	0.0006
GraphRAG	Basic (Naive)	0.775	1.96	4,599	0.0006	0.778	38.49	12,835	0.0014
GraphRAG	Local	0.773	4.27	7,866	0.0009	0.771	4.66	10,082	0.0011
GraphRAG	Global	0.714	162.66	643,909	0.0679	0.604	135.34	519,609	0.0533
GraphRAG	DRIFT	0.657	142.04	6,441,013	0.7901	0.572	207.66	33,160,141	3.9303
LightRAG	Local	0.762	3.17	16,083	0.0018	0.789	7.51	46,309	0.0048
LightRAG	Global	0.694	3.06	15,594	0.0017	0.747	8.45	38,204	0.0039
LightRAG	Mix	0.760	3.62	20,361	0.0022	0.796	7.85	88,449	0.0090
LightRAG	Hybrid	0.763	3.32	20,567	0.0022	0.791	7.59	69,480	0.0071
HiRAG	Hi	0.767	10.95	29,988	0.0031	0.783	6.15	28,200	0.0029
HiRAG	Local	0.777	8.03	14,296	0.0016	0.785	5.49	18,792	0.0020
HiRAG	Global	0.773	3.17	20,112	0.0021	0.790	6.54	20,177	0.0021
HiRAG	Bridge	0.760	2.68	16,631	0.0018	0.786	5.93	16,149	0.0017
HiRAG	No-Bridge	0.754	2.70	26,432	0.0028	0.782	6.13	29,549	0.0031
LinearRAG	—	0.784	8.01	3,077	0.0006	0.801	4.79	68,774	0.0071

Table 1: Main results comparing quality (Q_{total}), latency (seconds/query), token consumption, and cost (\$/query) across the DistComp D_{1280} benchmark and the UltraDomain Mix split.

overall quality while passing a more efficient final context to the generator.

The UltraDomain results show a similar pattern. LiteRAG and LinearRAG attain the highest overall quality (0.801), followed by LightRAG Mix (0.796). LiteRAG also records the lowest latency (1.24s), token consumption (4,862), and cost (\$0.0006) among all evaluated methods.

While LinearRAG matches LiteRAG’s aggregate quality on UltraDomain, it requires 4.79 seconds, 68,774 tokens, and \$0.0071 per query. Relative to LinearRAG, LiteRAG achieves a $14\times$ reduction in token volume, about a 92% reduction in cost, and a 74% reduction in latency. Other high-performing baselines incur even greater overhead; LightRAG Mix averages 7.85s and \$0.0090 per query, while GraphRAG DRIFT exceeds 200s and \$3.93.

Taken together, these results indicate that LiteRAG matches the quality of the strongest graph-based methods while using a much smaller execution budget. This is consistent with our central claim that a compact, structurally curated context can preserve answer quality without requiring the LLM to process large volumes of retrieved text.

5.2 Latency and Cost Across Corpus Scale

We next examine how efficiency changes as the DistComp corpus grows from 40 to 1280 documents. Figure 2 shows LiteRAG and the strongest baseline configuration from each method family, while Appendix A reports all configurations.

Latency Trends LiteRAG shows the lowest and most stable latency profile over the evaluated range,

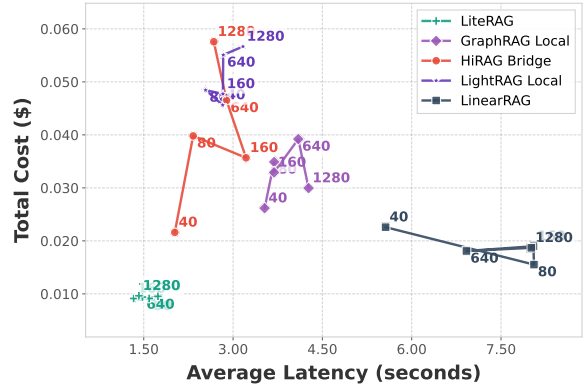


Figure 2: Latency-cost comparison across corpus sizes for LiteRAG and the best reported baseline configuration from each compared method family; labels indicate dataset size (D_{40} , D_{80} , D_{160} , D_{640} , and D_{1280}).

remaining between 1.33 and 1.74 seconds per query and ending at 1.42 seconds on D_{1280} . The selected baselines are consistently slower: LightRAG Local ranges from 2.54 to 3.17 seconds, HiRAG Bridge from 2.02 to 3.22 seconds, GraphRAG Local from 3.53 to 4.27 seconds, and LinearRAG from 5.56 to 8.05 seconds. The gap is present at every corpus size and becomes much larger for the more expensive graph-based modes reported in the appendix.

The appendix makes this separation clearer. GraphRAG Global grows from 8.57s (D_{40}) to 162.66s (D_{1280}), and DRIFT remains above 139s throughout. Over the same settings, LiteRAG stays within a narrow band. The reported results therefore show that LiteRAG preserves low end-to-end latency as corpus size increases, whereas broader graph aggregation and agentic traversal incur much

larger runtime overhead.

Cost Trends The cost results follow the same pattern. LiteRAG remains between \$0.00028 and \$0.00030 per query across all corpus sizes. The selected baselines remain more expensive: LinearRAG reaches \$0.0006 on D_{1280} , LightRAG Local ranges from \$0.0014 to \$0.0018, HiRAG Bridge from \$0.0007 to \$0.0018, and GraphRAG Local from \$0.0008 to \$0.0012. The appendix again shows the largest increases in the LLM-intensive configurations, with GraphRAG Global rising from \$0.0031 to \$0.0679 and GraphRAG DRIFT remaining above \$0.6195 throughout.

Taken together, these results show that LiteRAG maintains the lowest reported latency-cost profile over the evaluated range. This is consistent with the design in Section 3: bounded candidate selection, thresholded query-conditioned expansion, and compact context construction rather than corpus-level summarization or agentic traversal. Section 5.3 examines the same pattern under an explicit fixed-budget constraint.

5.3 Quality Under a Fixed Token Budget

This experiment evaluates answer quality when each method is constrained to a context budget of about 2,000 tokens, comparable to LiteRAG’s default setting. For each baseline, the budgeted variant tightens the retrieval configuration so that less context is passed to the generator while keeping the shared generation model unchanged.

LiteRAG is unchanged because its pipeline already meets the budget. Table 2 therefore shows no quality loss for LiteRAG, whereas nearly all baselines decline. The largest drops are LightRAG Global (−16.1%), GraphRAG DRIFT (−13.3%), LightRAG Local (−9.3%), and LightRAG Mix (−8.7%). GraphRAG Basic and Local drop by 5.2% and 4.0%, while LinearRAG declines by 2.1%. HiRAG is the most resilient baseline family, but its strongest configurations still remain below LiteRAG’s budgeted score.

GraphRAG Global could not be evaluated because its Map-Reduce retrieval procedure does not expose a comparable hard token cap. GraphRAG DRIFT is also informative: even its most restrictive available configuration still uses 570,440 tokens, down from 6,441,013, and still loses 13.3% in quality. Some retrieval pipelines therefore cannot be reduced to a small prompt budget without either substantial degradation or budget violation.

Method	Default	Budgeted	Δ (%)
<i>GraphRAG</i>			
Basic	77.5%	72.3%	−5.2%
Local	77.3%	73.3%	−4.0%
Global	71.4%	—	—
DRIFT	65.7%	52.4%	−13.3%
<i>LightRAG</i>			
Global	69.4%	53.3%	−16.1%
Hybrid	76.3%	71.9%	−4.4%
Local	76.2%	66.9%	−9.3%
Mix	76.0%	67.3%	−8.7%
<i>HiRAG</i>			
Hi	76.7%	72.9%	−3.8%
Local	77.7%	75.7%	−2.0%
Global	77.3%	73.6%	−3.7%
Bridge	76.0%	74.8%	−1.2%
No-Bridge	75.4%	74.7%	−0.7%
LinearRAG	78.4%	76.3%	−2.1%
LiteRAG	79.8%	79.8%	0.0%

Table 2: Default and budgeted Q_{total} under a fixed token budget of approximately 2,000 tokens; LiteRAG’s budgeted setting is identical to its default setting.

The contrast with LiteRAG is consistent with Phase 3. LiteRAG ranks the retrieved subgraph and constructs the final reasoning-chain context from the retained evidence, so the standard retrieval output already fits the target budget without an additional compression step.

Taken together, these results show that LiteRAG’s advantage is not only lower average token use, but also stronger quality retention under an explicit budget constraint. This complements the scalability analysis: the same bounded retrieval and context-construction stages that reduce latency and cost also preserve answer quality when context is tightly limited.

5.4 Ablation Study

To isolate the contribution of LiteRAG’s main design choices, we conduct an ablation study on 32 complex multi-hop queries. We systematically disable key pipeline components and measure the resulting changes in overall quality (Q_{total}), token consumption, and cost (Table 3).

Lexical and Community Signals (Phase 1): Removing the lexical and community signals from Phase 1 (*Semantic-only anchors* in Table 3) reduces overall quality from 0.798 to 0.761. This is consistent with the role of query-conditioned anchor selection: semantic similarity alone is less reliable for preserving exact entity terminology and for biasing retrieval toward relevant graph regions.

Configuration	$Q_{\text{total}} \uparrow$	Tokens $\downarrow$	Cost (\$) $\downarrow$
LiteRAG (Full)	0.798	2,291	0.00030
Semantic-only anchors	0.761	2,103	0.00027
Fixed-hop expansion	0.787	7,979	0.00087
No hub penalty	0.789	4,522	0.00052
Raw subgraph context	0.780	3,622	0.00043

Table 3: Ablation study evaluating the impact of core architectural components on overall quality, token efficiency, and cost per query.

Query-Adaptive Thresholding (Phase 2): Replacing query-adaptive thresholding with a rigid fixed-hop expansion (*Fixed-hop expansion* in Table 3) leaves overall quality relatively stable (0.787), but increases average token consumption by nearly 250% (from 2,291 to 7,979) and roughly triples per-query cost. This pattern suggests that fixed-hop traversal pulls in substantially more low-yield neighboring context. Within this ablation, query-adaptive thresholding is therefore the main contributor to LiteRAG’s token-efficiency.

Community-Aware Hub Penalization (Phase 2): Removing community-aware hub penalization (*No hub penalty* in Table 3) increases token consumption to 4,522 while leaving overall quality similar (0.789). This suggests that, without the penalty, high-degree hubs mainly add structural noise and cost rather than useful evidence.

Reasoning-Chain Context Construction (Phase 3): Bypassing reasoning-chain context construction (*Raw subgraph context* in Table 3) and instead passing the retrieved subgraph to the LLM as raw lists of entities and relationships increases token consumption by nearly 60% (3,622 tokens) and lowers overall quality to 0.780. This result is consistent with the role of Phase 3: structuring retained graph evidence into a more compact and directly usable final context.

6 Conclusion

In this paper, we presented LiteRAG, a graph-based retrieval method for RAG built on a simple premise: for multi-hop generation, retrieval should not only find relevant evidence, but deliver it to the LLM in a compact, structurally explicit form. LiteRAG addresses this objective by replacing LLM-mediated navigation with query-conditioned algorithmic exploration and reasoning-chain context construction. In particular, the method combines lexical and community-aware anchor selection, query-adaptive thresholding, community-aware hub penalization,

and compact reasoning-chain construction before final generation.

Across the reported experiments, this design yields a favorable quality-efficiency trade-off. On DistComp D_{1280} , LiteRAG attains the highest reported Q_{total} among the compared methods while also using the fewest tokens, the lowest latency, and the lowest per-query cost. The broader scaling results show a comparatively flat latency-cost profile over the evaluated corpus range, and the fixed-budget analysis suggests that LiteRAG’s advantage is not merely retrieving less context, but retrieving context with higher effective information density and remaining effective under a small budget. On UltraDomain, LiteRAG remains competitive in aggregate quality while using a substantially smaller retrieved context and markedly lower execution cost than the strongest comparator.

Taken together, these findings support the paper’s three main contributions. The comparative results establish the overall quality-efficiency gains; the fixed-budget and ablation results show that reasoning-chain context construction helps preserve overall quality under tight token budgets; and the ablation study further shows that lexical and community-aware anchor selection supports retrieval quality, while query-adaptive thresholding and community-aware hub penalization are the main drivers of token-efficiency. More broadly, separating graph exploration from LLM inference and treating context construction as a core part of retrieval yields a smaller, more information-dense final context for the generator. Future work can refine retrieval controls and test the same context-curation principle across additional domains.

Limitations

This study has some limitations. First, DistComp is a specialized benchmark centered on distributed-systems literature, so the absolute performance levels reported here should not be assumed to transfer unchanged to every domain; the UltraDomain results are intended to check that the main quality-efficiency pattern is not confined to that corpus. Second, LiteRAG focuses on retrieval and context construction rather than graph indexing, so the paper does not address the full end-to-end optimization problem when indexing cost is itself a primary concern. Third, all systems are evaluated under a shared generation and embedding setup to isolate retrieval behavior, which improves comparability

but leaves cross-model robustness for future work. Finally, the scalability analysis is empirical rather than formal, and some baseline architectures do not expose controls that permit perfectly matched hard token budgets across methods. These limitations primarily affect scope and generality, rather than the paper’s central claim that LiteRAG delivers a strong quality-efficiency trade-off through query-conditioned graph exploration and compact context construction.

References

- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva N. Mody, Steven Truitt, and Jonathan Larson. 2024. [From local to global: A graph rag approach to query-focused summarization](#). *ArXiv*, abs/2404.16130.
- Wenqi Fan, Yajuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. [A survey on rag meeting llms: Towards retrieval-augmented large language models](#). In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD ’24, pages 6491–6501, New York, NY, USA. Association for Computing Machinery.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jin Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2023. [Retrieval-augmented generation for large language models: A survey](#). *ArXiv*, abs/2312.10997.
- Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. [LightRAG: Simple and fast retrieval-augmented generation](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10746–10761, Suzhou, China. Association for Computational Linguistics.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: neurobiologically inspired long-term memory for large language models. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS ’24, Red Hook, NY, USA. Curran Associates Inc.
- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. [G-retriever: Retrieval-augmented generation for textual graph understanding and question answering](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Haoyu Huang, Yongfeng Huang, Yang Junjie, Zhenyu Pan, Yongqiang Chen, Kaili Ma, Hongzhi Chen, and James Cheng. 2025a. [Retrieval-augmented generation with hierarchical knowledge](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 6044–6060, Suzhou, China. Association for Computational Linguistics.
- Yiqian Huang, Shiqi Zhang, and Xiaokui Xiao. 2025b. [Ket-rag: A cost-efficient multi-granular indexing framework for graph-rag](#). In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD ’25, page 1003–1012, New York, NY, USA. Association for Computing Machinery.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023a. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376.
- Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqiu Sun, Qian Liu, Jane Dwyer, Graham Neubig, Baolin Peng, and Dragomir Kalai. 2023b. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS ’20, Red Hook, NY, USA. Curran Associates Inc.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.
- Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *The Twelfth International Conference on Learning Representations*.
- Zhefan Wang, Huanjun Kong, Jie Ying, Wanli Ouyang, and Nanqing Dong. 2025. [ROGRAG: A robustly optimized GraphRAG framework](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 604–613, Vienna, Austria. Association for Computational Linguistics.
- Luyao Zhuang, Shengyuan Chen, Yilin Xiao, Huachi Zhou, Yujing Zhang, Hao Chen, Qinggang Zhang, and Xiao Huang. 2026. [LinearRAG: Linear graph retrieval augmented generation on large-scale corpora](#). In *The Fourteenth International Conference on Learning Representations*.

A Complete Scalability Results

Table 4 reports the complete scalability results for all evaluated query engines across all dataset sizes, expanding on the summary provided in Section 5.2. The table details average end-to-end latency, total token consumption, cost per query, and overall quality.

Engine	D	Lat. (s)	Tok.	Cost (\$)	Q_{total}	Engine	D	Lat. (s)	Tok.	Cost (\$)	Q_{total}
GraphRAG Basic	40	2.13	4,417	0.0006	0.8067	HiRAG Hi	40	5.92	11,319	0.0013	0.7761
	80	1.62	4,509	0.0006	0.7938		80	2.32	23,914	0.0025	0.7706
	160	1.56	4,354	0.0006	0.8213		160	3.92	22,453	0.0024	0.8049
	640	1.84	4,539	0.0006	0.7906		640	2.99	26,958	0.0028	0.7841
	1280	1.96	4,599	0.0006	0.7746		1280	10.95	29,988	0.0031	0.7668
GraphRAG Local	40	3.53	6,482	0.0008	0.7547	HiRAG Local	40	2.26	6,640	0.0008	0.7680
	80	3.69	9,331	0.0011	0.7818		80	1.96	8,833	0.0010	0.7860
	160	3.69	8,739	0.0010	0.7152		160	8.77	8,029	0.0009	0.8115
	640	4.10	10,630	0.0012	0.7610		640	8.24	10,834	0.0012	0.7924
	1280	4.27	7,866	0.0009	0.7727		1280	8.03	14,296	0.0016	0.7768
GraphRAG Global	40	8.57	26,633	0.0031	0.7077	HiRAG Global	40	2.39	7,764	0.0009	0.7630
	80	19.16	53,208	0.0059	0.7498		80	2.16	15,935	0.0017	0.8069
	160	30.63	93,364	0.0100	0.7156		160	8.06	15,164	0.0016	0.8059
	640	92.68	348,525	0.0365	0.7088		640	5.35	17,540	0.0019	0.7905
	1280	162.66	643,909	0.0679	0.7144		1280	3.17	20,112	0.0021	0.7731
GraphRAG DRIFT	40	142.90	5,103,967	0.6470	0.6837	HiRAG Bridge	40	2.02	5,584	0.0007	0.7210
	80	139.83	4,832,192	0.6195	0.5861		80	2.33	11,362	0.0012	0.7780
	160	147.81	6,111,591	0.7502	0.6159		160	3.22	10,035	0.0011	0.8047
	640	149.29	7,312,642	0.8777	0.4488		640	2.89	13,245	0.0015	0.7812
	1280	142.04	6,441,013	0.7901	0.6574		1280	2.68	16,631	0.0018	0.7602
LightRAG Global	40	3.00	13,178	0.0015	0.7932	HiRAG No-Bridge	40	2.35	11,330	0.0013	0.7760
	80	3.08	13,624	0.0015	0.7657		80	2.23	20,305	0.0022	0.7915
	160	2.50	13,859	0.0015	0.7803		160	4.43	19,425	0.0021	0.8050
	640	3.09	15,557	0.0017	0.7524		640	3.52	22,890	0.0024	0.7788
	1280	3.06	15,594	0.0017	0.6943		1280	2.70	26,432	0.0028	0.7541
LightRAG Hybrid	40	3.52	17,246	0.0019	0.7939	LinearRAG	40	5.56	3,266	0.0007	0.8081
	80	3.28	18,491	0.0020	0.7988		80	8.05	2,649	0.0005	0.7826
	160	3.34	18,465	0.0020	0.8103		160	8.05	2,800	0.0006	0.8223
	640	3.63	20,256	0.0022	0.7877		640	6.92	2,904	0.0006	0.7484
	1280	3.32	20,567	0.0022	0.7627		1280	8.01	3,077	0.0006	0.7841
LightRAG Local	40	2.83	12,509	0.0014	0.7956	LiteRAG	40	1.74	2,158	0.0003	0.8090
	80	2.54	13,650	0.0015	0.7969		80	1.59	2,195	0.0003	0.8277
	160	2.83	13,364	0.0015	0.8179		160	1.33	2,170	0.0003	0.8398
	640	2.83	15,566	0.0017	0.8112		640	1.48	2,235	0.0003	0.8090
	1280	3.17	16,083	0.0018	0.7622		1280	1.42	2,291	0.0003	0.7980
LightRAG Mix	40	3.71	17,126	0.0019	0.8010						
	80	3.39	18,292	0.0020	0.7991						
	160	3.09	18,573	0.0020	0.8044						
	640	3.70	20,166	0.0022	0.7895						
	1280	3.62	20,361	0.0022	0.7596						

Table 4: Complete scalability results for all query engines across all DistComp dataset sizes (D), detailing end-to-end latency, token consumption, cost per query, and Q_{total} .

B Disaggregated Quality Metrics

This appendix reports the disaggregated components of the composite quality metric Q_{total} for the top-performing configurations on the 1280-document corpus.

The composite metric is defined in Equation 9, with weights 0.6, 0.25, and 0.15 assigned to the judge-based, semantic, and lexical components, respectively (as introduced in Section 4.4). The weights were fixed on a held-out validation split after comparing candidate mixtures of judge-based, semantic, and lexical signals. The largest weight is assigned to Q_{judge} because it is the only component that directly evaluates multi-hop reasoning quality, factual correctness, and question relevance in an integrated way. In our implementation, Q_{judge} is normalized to the $[0, 1]$ range from the aggregate scores of Correctness,

Completeness, and Relevance. Q_{sem} provides a softer semantic alignment signal, while Q_{lex} (ROUGE-L) acts as a stricter lexical grounding term that penalizes severe wording-level drift or unsupported terminology.

Table 5 displays the raw scores for each sub-metric. LiteRAG demonstrates consistently high performance across all three independent evaluators, suggesting that its efficiency (detailed in Section 5.1) does not come at the cost of semantic or factual degradation.

System (Mode)	Q_{judge}	Q_{sem}	Q_{lex}	Q_{total}
LiteRAG (Default)	0.909	0.866	0.242	0.798
GraphRAG (Basic)	0.904	0.857	0.119	0.775
GraphRAG (Drift)	0.732	0.807	0.111	0.657
GraphRAG (Global)	0.822	0.827	0.095	0.714
GraphRAG (Local)	0.912	0.837	0.108	0.773
LightRAG (Global)	0.774	0.827	0.155	0.694
LightRAG (Hybrid)	0.883	0.850	0.138	0.763
LightRAG (Local)	0.879	0.852	0.144	0.762
LightRAG (Mix)	0.877	0.850	0.137	0.760
HiRAG (Bridge)	0.881	0.852	0.124	0.760
HiRAG (Global)	0.907	0.854	0.102	0.773
HiRAG (Hi)	0.894	0.852	0.117	0.767
HiRAG (Local)	0.912	0.852	0.113	0.777
HiRAG (No-Bridge)	0.875	0.849	0.110	0.754
LinearRAG (Default)	0.851	0.875	0.365	0.784

Table 5: Disaggregated scores for LLM-as-a-judge (Q_{judge}), semantic similarity (Q_{sem}), lexical overlap (Q_{lex}), and the composite metric (Q_{total}) on the D_{1280} benchmark.

C Prompt Templates

To ensure reproducibility, we provide the exact prompt templates used for context assembly, final answer generation, and the LLM-as-a-judge evaluation. These templates were extracted directly from the system’s source code.

C.1 LiteRAG Context Assembly and Generation Prompts

As described in Section 3.3, LiteRAG avoids dumping raw text by converting graph relationships into highly dense, structured sections. During the final response generation phase, the system prompt and user prompt are constructed as follows:

System Prompt

You are a helpful assistant that answers questions based on provided knowledge graph context. Be accurate, cite specific entities when relevant, and acknowledge if information is incomplete.

User Prompt

```
## Knowledge Graph Context
[Direct Evidence (Source Text)]
[Graph Context]: ...

[Graph Reasoning Chains (Connections)]
• **Entity A** is connected to **Entity B**
  via *Description of the relationship*
...

[Entity Definitions]
**Entity A**: Description of Entity A...
...

## Question
{user_query}

## Answer
Based on the knowledge graph context above, provide a clear and accurate answer:
```

By pre-computing the logical connections in the *Graph Reasoning Chains* section, the prompt makes relationships explicit and reduces the need for the generator to infer them from dispersed passages, which lowers token consumption and the inferential burden on the generator.

C.2 LLM-as-a-judge Evaluation Prompt

For the judge-based evaluation (Q_{judge}), the independent evaluator utilizes a detailed, criteria-based prompt designed to enforce rigorous, step-by-step reasoning. The exact prompt used in the benchmarking framework is as follows:

Evaluation Prompt

You are an impartial and scientific evaluator. Your task is to assess a generated answer against a ground truth reference, based on a specific question. Evaluate the answer based on the criteria of Correctness, Completeness, and Relevance.

****Evaluation Task:****

1. Analyze the Question: Understand what the user is asking for.
 - Question: "{question}"
2. Analyze the Ground Truth: This is the reference for what a correct and complete answer should contain.
 - Ground Truth: "{ground_truth}"
3. Analyze the Generated Answer: This is the answer you must evaluate.
 - Generated Answer: "{prediction}"
4. Perform a Step-by-Step Evaluation:
 - Correctness (0.0-1.0): Is the information in the Generated Answer factually accurate and consistent with the Ground Truth? Does it contradict the ground truth or introduce plausible but unsupported information (hallucinations)?
 - Completeness (0.0-1.0): Does the Generated Answer cover all the key information and essential points present in the Ground Truth?
 - Relevance (0.0-1.0): Does the Generated Answer directly address the user's Question? Is the answer on-topic?
5. Provide Scores and Reasoning: Based on your analysis, provide a score from 0.0 (terrible) to 1.0 (perfect) for each criterion and write a detailed reasoning for your scores.

Return your evaluation as JSON with this exact format:

```
{
  "correctness": <float 0-1>,
  "completeness": <float 0-1>,
  "relevance": <float 0-1>,
  "reasoning": "<string>"
}
```

D LiteRAG Hyperparameter Configuration

This appendix reports the exact LiteRAG configuration used in all experiments. As noted in Section 4, hyperparameters were selected by grid search on a held-out validation split under the same model and graph-construction setup used for the reported results.

Table 6 instantiates the parameters introduced in the LiteRAG method (Section 3). For Phase 1 (Section 3.1), the lexical term S_{lex} is implemented as a combination of exact and fuzzy matching, so the table decomposes the single lexical weight β in Equation 1 into its implementation-level components. For Phase 2 (Section 3.2), the reported runs used an explicit hop-depth budget $k_{\text{max}} = 3$ together with a per-anchor expansion cap $N_{\text{max}} = 50$, so exploration was governed by the dynamic thresholding rule under bounded traversal budgets.

These settings were used for every LiteRAG result reported in the main paper. Together, they reflect the intended retrieval bias of the method: preserve exact lexical anchors when necessary, expand conservatively from strong initial evidence, and rank Phase 3 (Section 3.3) entities primarily by repeated support across anchor-induced traversals rather than by generic graph centrality alone.

Phase	Parameter	Value	Description
Anchor Selection	candidate_pool_size (K)	8	Number of top vector-retrieved candidates retained before anchor scoring. This keeps Phase 1 local to a small semantically relevant pool rather than scoring the full graph.
	min_anchor_score (τ_{anchor})	0.30	Minimum composite anchor score required for a node to enter A . The selected value preserves multiple plausible entry points without flooding Phase 2 with weak anchors.
	semantic_weight (α)	0.40	Contribution of semantic similarity to the Phase 1 anchor score.
	lexical_weight (β)	0.45	Total contribution of lexical evidence to the Phase 1 anchor score. This relatively large weight protects exact terminology and named entities that may be poorly captured by embeddings alone.
	keyword_exact_weight	0.30	Exact-match component of S_{lex} . This is the dominant share of β and rewards direct terminology overlap.
	keyword_fuzzy_weight	0.15	Fuzzy-match component of S_{lex} , used to recover near matches and minor surface-form variations without overpowering exact lexical evidence.
	community_weight (γ)	0.15	Contribution of community-level relevance to the Phase 1 anchor score. The smaller weight keeps community evidence supportive rather than dominant.
Subgraph Expansion	max_exploration_depth (k_{max})	3	Maximum hop distance from each anchor during Phase 2. This bounds query-time search radius and is the primary explicit exploration budget used in the reported runs.
	max_expansions_per_anchor (N_{max})	50	A per-anchor expansion cap of 50 nodes was used to prevent unbounded exploration from highly connected anchors.
	min_relevance_threshold (τ_{base})	0.25	Base floor in the dynamic threshold $\tau_{\text{dyn}}(q, A)$. This prevents traversal from expanding on very weak evidence even when anchors are modest.
	signal_amplification (λ)	0.25	Scaling factor that raises the traversal threshold when the initial anchors are strong. The selected value makes expansion more selective for well-grounded queries while still allowing broader search when anchor quality is weaker.
	relevance_decay_factor (d)	0.70	Multiplicative decay applied per hop in $R(q, u, v, k)$. This encourages shorter explanatory paths while still permitting multi-hop retrieval.
	degree_influence (δ)	0.05	Strength of the hub penalty. The small value suppresses generic hubs without over-penalizing structurally important nodes.
	community_cohesion (κ)	0.80	Within-community protection factor in $\omega(u, v)$. This substantially relaxes hub penalization for transitions that remain inside the same topical region.
Consensus Ranking	intersection_weight (ρ_1)	0.30	Weight on $I_{\text{rate}}(v)$, rewarding entities reached by multiple anchor expansions. This is the largest single Phase 3 weight because repeated recovery across paths is treated as the strongest indicator of relevance.
	semantic_weight (ρ_2)	0.25	Weight on semantic alignment in the final ranking. This keeps the retained context tightly tied to the user query after graph expansion.
	proximity_weight (ρ_3)	0.25	Weight on topological proximity to the anchor set. Matching the semantic term, it favors nearby evidence without excluding informative multi-hop entities.
	structural_weight (ρ_4)	0.20	Weight on structural centrality. This is the smallest Phase 3 weight so that generic graph prominence does not override query-conditioned evidence.

Table 6: Exact LiteRAG hyperparameters used in the reported experiments. The table instantiates the abstract parameters from Section 3 and records the implementation-level decomposition of the lexical anchor term.

E Detailed Engine Configurations

This appendix provides the full configuration parameters for all retrieval engines evaluated in this study. To ensure readability and clear side-by-side comparison, we present the parameter configurations using standard academic tables. We provide both the standard settings used for the primary benchmarking (Table 1) and the budgeted settings used for the fixed-budget analysis (Section 5.3).

The default configurations used in this study were obtained from the standard parameters in the repositories of the different architectures.

E.1 LiteRAG Configuration

The LiteRAG configuration and hyperparameters are defined in Appendix D. LiteRAG does not require a separate budgeted configuration because its algorithmic design naturally produces a context within the 2,000-token target budget.

E.2 LinearRAG Configuration

The configuration for LinearRAG follows the suggested defaults for document-based retrieval as per its reference implementation.

Parameter	Standard	Budgeted
spacy_model	"en_core_web_trf"	
max_workers	4	4
retrieval_top_k	5	3
max_iterations	3	2
top_k_sentence	1	1
passage_ratio	1.5	1.5
iteration_threshold	0.5	0.5
use_vectorized	false	false

Table 7: LinearRAG configuration comparison.

E.3 LightRAG Configuration

To test LightRAG under a fixed budget, we reduced the `max_total_tokens` and associated entity/relation limits.

Parameter	Standard	Budgeted
top_k	60	60
max_entity_tokens	6000	600
max_relation_tokens	8000	600
max_total_tokens	30000	2000
enable_rerank	false	false

Table 8: LightRAG configuration comparison.

E.4 HiRAG Configuration

For the budgeted version, we enforced `limit_tokens: true` and capped the total context at 2,000 tokens.

E.5 Microsoft GraphRAG Summary

The budgeted version capped `max_context_tokens` at 4,000 (the lowest functional setting for DRIFT) and reduced the basic search `k`.

F Exploratory Multi-Entity Efficiency Comparison

As an exploratory analysis extending the findings in Section 5, this appendix reports a focused comparison between LiteRAG and LinearRAG on a small DistComp subset designed to probe multi-entity queries. The subset contains 20 queries, with 5 queries in each of four bins defined by the number of distinct

Parameter	Standard	Budgeted
top_k	20	20
top_m	10	10
max_token_text_unit	20000	—
max_token_local	20000	—
max_token_bridge	12500	—
max_token_report	12500	—
max_total_tokens	—	2000
limit_tokens	false	true

Table 9: HiRAG configuration comparison.

Mode / Parameter	Standard	Budgeted
<i>Local Search</i>		
max_context_tokens	12000	4000
<i>Global Search</i>		
max_context_tokens	12000	4000
<i>DRIFT Search</i>		
n_depth	2	—
concurrency	32	—
loc_search_max_data_toks	—	4000
<i>Basic Search</i>		
k	—	4
max_context_tokens	—	4000

Table 10: GraphRAG configuration comparison.

entities or concepts explicitly mentioned in the question. The purpose of this analysis is focused: to examine whether the efficiency gap between the two methods remains visible as the number of entities in the query increases.

Ent.	L-Lat.	R-Lat.	L-Tok.	R-Tok.	L-Cost	R-Cost
1	1.02	2.92	2,009	2,365	0.00022	0.00033
2	1.11	2.59	2,120	3,077	0.00024	0.00044
3	1.00	2.12	1,881	2,475	0.00020	0.00034
4	1.09	3.77	1,961	2,833	0.00022	0.00048

Table 11: Efficiency comparison on the 20-query multi-entity subset. **L** denotes LiteRAG and **R** denotes LinearRAG. Latency is reported in seconds per query and cost in dollars per query.

Across all four bins in Table 11, LiteRAG remains faster, less expensive, and more token-efficient than LinearRAG. LiteRAG stays between 1.00 and 1.11 seconds and between 1,881 and 2,120 tokens, whereas LinearRAG ranges from 2.12 to 3.77 seconds and from 2,365 to 3,077 tokens. The quality differences on this subset are mixed in the lower-entity bins, while LiteRAG records the higher score on the 4-entity bin (0.81 vs. 0.74). Accordingly, the clearest conclusion from this subset is that LiteRAG preserves a more stable efficiency profile across query-complexity bins.

G Example Queries by Entity Complexity

This appendix provides representative examples from the multi-entity subset summarized in Appendix F. The queries are grouped by the number of distinct entities that must be related to produce a correct answer.

1 Entity (Simple Lookup)

- “What specific hardware mechanism does ‘Poseidon’ use to protect heap metadata?”
- “What is the primary function of the ‘IoTwins’ platform?”

2 Entities (Relational Reasoning)

- *“How does ‘SparkFlow’ utilize ‘Base Recalibrator’ in its design?”*
- *“In the context of binary hardening, what capability did ‘RedFat’ demonstrate regarding ‘Google Chrome’?”*

3 Entities (Multi-hop Synthesis)

- *“What is the relationship between ‘HopsFS-CL’, ‘HopsFS’, and ‘HDFS’?”*
- *“How does ‘Siren’ improve upon current ‘Byzantine-robust’ aggregation rules in ‘Federated Learning’?”*

4 Entities (Comparative Analysis)

- *“In the scalability evaluation of deep learning frameworks, which framework among ‘Tensorflow’, ‘Keras’, ‘MXNet’, and ‘PyTorch’ was found to be the most efficient?”*
- *“How much did ‘EndGraph’ improve preprocessing performance compared to the group of ‘LFGraph’, ‘PowerLyra’, ‘PowerGraph’, and ‘D-Galois’?”*